1. Rationale

Target Market: individual vehicle owners and automotive enthusiasts who want meticulous control of their service records

The Need: Dealerships and shops often keep fragmented records. For the owners of modern vehicles like a Kia Sportage Hybrid, maintaining an exact, centralized log of mileage, service dates, and warranty documentation is critical for preserving vehicle value and performance. Relying exclusively on paper files is messy and commercial software can often be too complex or expensive for a single user.

2. Description:

A desktop application built with a JavaFX graphical user interface and a Java Spring Boot backend, utilizing a MySQL database for complete data persistence. The system allows a user to comprehensively manage vehicle upkeep by logging new service events, modifying existing records, or deleting inaccurate entries. Additionally, users can attach digital files (such as PDF shop receipts or scanned invoices) to specific logs using local file directory storage. The application provides a centralized table view to easily retrieve historical records, update details, and open associated documents, ensuring a highly maintainable and accessible vehicle history.

3. Functional and Nonfunctional Requirements

Functional Requirements:

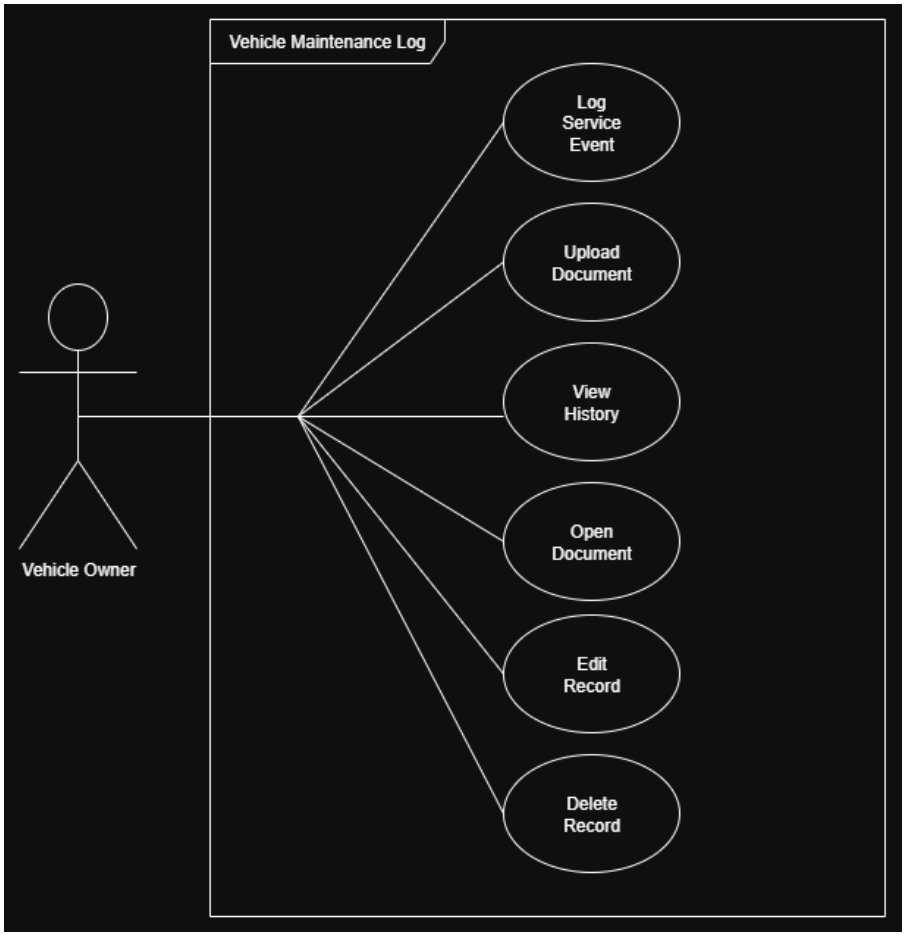
- The system shall allow a user to input a new maintenance record including date, current mileage, and service description.
- The system shall allow the user to utilize a file directory window to select and attach a local digital document (PDF, Image, or Doc).
- The system shall save the file path of the uploaded document to the database.
- The system shall display a table view of all historical maintenance records.

- The system shall allow the user to open an attached receipt or document directly from the table view.
- The system shall allow the user to modify the details of an existing maintenance record.
- The system shall allow the user to permanently delete an existing maintenance record from the database.

Non Functional Requirements:

- The graphical user interface must be built using JavaFX.
- The backend architecture must utilize Java Spring Boot.
- Data Persistence must be handled by a MySql database.
- The system must run locally on a desktop environment.

4. Use Case Diagram:



5. Use Case Description:

Use Case: Log Service Event

1. The Vehicle Owner enters the date, mileage, and service details into the JavaFX form.
2. The Vehicle Owner clicks Save.
3. The system validates that the fields are not empty.
4. The system transmits the data to the Spring Boot controller.
5. The system saves the new record to the MySQL database.
6. The system refreshes the user interface to display the newly added record in the history table.

Use Case: Edit Maintenance Record

1. The Vehicle Owner selects a specific maintenance record from the history table view.
2. The Vehicle Owner clicks the Edit button.
3. The system populates the data entry form with the existing details of the selected record (Date, Mileage, Service Description).
4. The Vehicle Owner modifies the desired fields and clicks Save.
5. The system validates that the modified fields are not empty.
6. The system transmits the updated data to the Spring Boot backend.
7. The Spring Boot backend updates the specific record in the MySQL database.
8. The system refreshes the JavaFX table view to display the updated information.

Use Case: Delete Maintenance Record

1. The Vehicle Owner selects a specific maintenance record from the history table view.
2. The Vehicle Owner clicks the Delete button.
3. The system displays a confirmation dialog asking the user to verify the deletion.
4. The Vehicle Owner clicks Confirm.
5. The system sends a delete request to the Spring Boot backend for that specific record.
6. The Spring Boot backend permanently removes the record from the MySQL database.
7. The system refreshes the JavaFX table view, removing the deleted record from the display.

Use Case: Upload Document

1. The Vehicle Owner clicks the Upload Document button on the data entry form.
2. The system opens a native file directory window.

3. The Vehicle Owner navigates to and selects a local digital document (PDF or Doc).
4. The system copies the selected file into a dedicated local application folder.
5. The system generates a string representing the local file path.
6. The system associates this file path string with the current maintenance record being created.

Use Case: View Maintenance History

1. The Vehicle Owner launches the application or navigates to the main dashboard.
2. The system sends a request to the Spring Boot backend to retrieve all existing maintenance records.
3. The Spring Boot backend queries the MySQL database for the records.
4. The database returns the saved records to the backend.
5. The system populates the JavaFX table view with the retrieved data, displaying columns such as Date, Mileage, and Service Description.

Use Case: Open Attached Document

1. The Vehicle Owner selects a specific maintenance record from the history table view.
2. The Vehicle Owner clicks the View Document button.
3. The system retrieves the saved local file path string associated with that specific record.
4. The system verifies that the file exists at the specified local path.
5. The system commands the computer's default file viewer (ex PDF reader or image viewer) to open the document for the user to read.